AnimeIndex

Catálogo de Series — Briefing de Proyecto

Programación 4 · Frontend

---

1. Descripción general

AnimeIndex es una plataforma web que permite explorar un catálogo de series, consultar su estado de emisión y revisar sus puntuaciones, consumiendo datos en tiempo real desde una API pública.

2. Objetivo

Construir una interfaz de usuario (UI) que permita buscar, listar y validar información de un catálogo de series, manejando de forma robusta las limitaciones de la API externa (rate limiting) sin degradar la experiencia del usuario. El foco del ejercicio está en el manejo de estado, peticiones asíncronas y experiencia de usuario ante errores, más que en el contenido específico que se muestra.

3. Fuente de datos (API)

- **Endpoint:** https://api.jikan.moe/v4/anime?q={query}&limit={n}
- **Ejemplo:** https://api.jikan.moe/v4/anime?q=naruto&limit=12
- **Documentación:** Jikan API v4 (wrapper no oficial de MyAnimeList)
- **Restricción crítica:** máximo 3 peticiones por segundo. Al superarlo, la API responde con HTTP 429 (Too Many Requests).

4. Datos a renderizar por cada elemento del catálogo

Campo	Origen en la respuesta	Notas
Título	title	Puede usarse title_english como alternativa
Imagen	images.jpg.image_url	Usar placeholder si falla la carga
Episodios	episodes	Puede venir null si sigue en emisión
Estado	status	Ej: "Currently Airing", "Finished Airing"
Género	genres[].name	Es un array; mostrar como tags/chips
Sinopsis	synopsis	Truncar con "ver más" si es muy larga
Puntuación	score	Puede venir null

5. Funcionalidades clave (alcance funcional)

- **Buscador:** input de texto para consultar series por nombre.
- **Listado/Grid:** tarjetas con imagen, título, estado y puntuación.
- **Validación de emisión:** indicador visual (badge/color) si está "En emisión" vs "Finalizado".

- **Detalle:** sinopsis, géneros y episodios al hacer clic o expandir.
- **Manejo de errores 429:** sistema de reintento + notificación no intrusiva (toast).

6. El reto técnico central: Rate Limiting (429)

Este es el punto que evalúa el dominio de manejo de estado asíncrono en frontend. La solución debe contemplar:

- **Debounce en el buscador:** evitar disparar una petición por cada tecla (esperar ~400–500ms de inactividad).
- **Cola o throttling de peticiones:** limitar las llamadas salientes a máx. 3/segundo (p. ej. con una cola simple o librerías como p-throttle / p-queue).
- **Interceptor de errores:** detectar el status 429 en la capa de fetch/axios antes de que rompa la UI.
- **Retry con backoff:** reintentar la petición fallida tras una breve espera (ej. exponential backoff: 1s, 2s, 4s).
- **Toast amigable:** mostrar un mensaje tipo "Estamos consultando muchos datos, espera un momento..." en vez de un error crudo, y sin bloquear la interfaz.
- **Estado de carga:** skeletons o spinners mientras se resuelve el reintento.

7. Arquitectura sugerida

- **Capa de servicio (api/jikan.js):** centraliza fetch, throttling e interceptor de 429.
- **Capa de estado:** hooks (useCatalogSearch) o store (Zustand/Redux) para manejar loading, data, error.
- **Capa de UI:** componentes desacoplados — SearchBar, ItemCard, CatalogGrid, Toast, DetailModal.
- **Sistema de notificaciones:** componente Toast/ToastContainer reutilizable, independiente de la lógica de negocio.

8. Criterios de aceptación

☐ La búsqueda no dispara peticiones excesivas al escribir rápido (debounce funcional).

☐ Al forzar un 429 (ej. múltiples búsquedas seguidas), la app no se rompe ni muestra pantalla en blanco.

☐ Se notifica el error 429 mediante un toast, no una alerta bloqueante ni un console.error silencioso.

☐ La app reintenta automáticamente la petición fallida.

☐ Todos los campos de datos (título, imagen, episodios, estado, género, sinopsis, puntuación) se renderizan correctamente, incluso con valores null.

☐ Diseño responsivo (grid adaptable).

9. Tecnologías sugeridas

- **Framework:** React / Vue (según lo visto en el curso)
- **HTTP client:** fetch nativo o Axios
- **Manejo de throttling:** p-throttle, p-limit, o cola manual
- **Notificaciones:** react-hot-toast, sonner, o componente propio
- **Estilos:** CSS Modules / TailwindCSS

10. Riesgos identificados

- Dependencia de una API pública gratuita: puede tener caídas o límites cambiantes.
- Datos inconsistentes (campos null) que pueden romper el render si no se validan.
- Rate limit compartido por IP: si varios estudiantes prueban desde la misma red, puede saturarse más rápido.